

Unit 3: 동기화 기법

강사: 조영민

목표

- Race Condition의 원인을 이해하고 lock 으로 해결할 수 있다
- Deadlock 발생 조건 4가지를 설명하고 예방 전략을 적용할 수 있다
- Monitor , Mutex , Semaphore , Interlocked , SpinLock 등 동기화 프리미티브의 차이와 적용 기준을 판단할 수 있다
- AutoResetEvent / ManualResetEvent 로 스레드 간 신호를 구현할 수 있다
- ConcurrentQueue 와 Actor 패턴으로 thread-safe (CAS 기반) 설계를 경험한다

동기화 프리미티브 분류 체계 (1/2)

분류	프리미티브	핵심 특징
상호 배제	lock , Monitor , Mutex	한 번에 하나의 스레드만 진입
신호 (Signal)	AutoResetEvent , ManualResetEvent	스레드 간 이벤트 알림
원자 연산 / 스핀	Interlocked , SpinLock	Interlocked 는 lock-free 원자 연산, SpinLock 은 사용자 모드 스핀 대기

동기화 프리미티브 분류 체계 (2/2)

분류	프리미티브	핵심 특징
동시 컬렉션	<code>ConcurrentQueue<T></code> 등	CAS 기반 Thread-Safe 자료구조
메시지 패싱	<code>Channel<T></code> , Actor 패턴	공유 상태 없이 메시지로 통신

- 신호(Signal) 분류에는 `Monitor.Pulse` 도 포함된다
- 상황에 맞는 프리미티브를 선택하는 것이 핵심이다

Race Condition -- 왜 발생하는가

`_value++` 는 원자적 연산이 아니다 (읽기 -> 증가 -> 쓰기, 3단계)

```
// Thread-Unsafe: 두 스레드가 동시에 실행하면 결과가 매번 다르다  
public void IncUnsafe() => _value++;
```

두 스레드가 100만 회씩 증가시키면 예상값은 200만이지만, 실제로는 그보다 작은 값이 나온다. **읽기와 쓰기 사이에 다른 스레드가 끼어들기 때문이다.**

Race Condition -- 면접 포인트

면접 TIP: "Race Condition이란 무엇인가?" -- 두 개 이상의 스레드가 공유 데이터를 동시에 읽기/쓰기할 때, 실행 순서에 따라 결과가 달라지는 버그를 말한다.

- `_value++` 는 3단계(읽기 -> 증가 -> 쓰기)로 분해된다
- 스레드 간 실행 순서는 OS가 결정하므로 예측 불가능하다
- 해결 핵심: **임계 구역(Critical Section)** 을 보호한다

lock 키워드 -- Monitor.Enter/Exit

```
private readonly object _gate = new();  
public void IncSafe()  
{  
    lock (_gate) { _value++; }  
}
```

- 컴파일러가 `lock(obj)` 를 `Monitor.Enter(obj, ref lockTaken) + try/finally + if (lockTaken) Monitor.Exit(obj)` 로 변환 (C# 4.0+ lockTaken 패턴)
- 예외가 발생해도 락 해제가 보장된다
- 두 스레드가 각 100만 회 증가 -> 결과는 항상 정확히 200만

lock 사용 주의사항

잘못된 패턴	위험	이유
<code>lock(this)</code>	데드락	외부에서 동일 인스턴스에 lock 가능
<code>lock("string")</code>	전역 교착	문자열 인터닝 으로 같은 객체 공유
<code>lock(typeof(T))</code>	AppDomain 교착	Type 객체는 전체에서 단일 인스턴스

올바른 방법: 항상 `private readonly object _gate = new();` 사용

lock 주의사항 -- 면접 포인트

면접 TIP: "lock(this)를 쓰면 안 되는 이유는?" -- 외부 코드가 같은 객체에 lock을 걸어 의도치 않은 데드락이 발생할 수 있다. 전용 private 객체를 사용해야 한다.

- `lock("string")` 도 위험 -- 문자열 인터닝으로 프로그램 전체가 같은 참조를 공유
- 항상 전용 private readonly object를 락 대상으로 사용한다

Deadlock -- 4가지 조건과 해결법

데드락 발생 4가지 필요 조건 (모두 충족 시 발생):

1. 상호 배제 -- 자원을 한 번에 하나만 사용 가능
2. 점유 대기 -- 자원을 가진 채 다른 자원을 요청
3. 비선점 -- 다른 스레드의 자원을 강제로 뺏을 수 없음
4. 순환 대기 -- A -> B -> A 형태로 서로를 기다림

해결: 모든 스레드가 동일한 순서로 락을 획득하면 순환 대기 조건이 깨진다.

Deadlock -- 코드 예시

```
// 데드락: T1은 LockX->LockY, T2는 LockY->LockX  
var t1 = new Thread(() => Work(LockX, LockY, "T1"));  
var t2 = new Thread(() => Work(LockY, LockX, "T2"));
```

```
// 해결: 모든 스레드가 LockX -> LockY 순서로 획득  
var t1 = new Thread(() => Work(LockX, LockY, "T1"));  
var t2 = new Thread(() => Work(LockX, LockY, "T2"));
```

- 락 순서를 통일하면 **순환 대기** 조건이 제거된다

Monitor.Wait / Pulse -- Writer

```
static void Writer(object? obj)
{
    lock (Gate)
    {
        Queue.Enqueue((int)obj!);
        Monitor.Pulse(Gate); // 대기 중인 Reader를 깨움
    }
}
```

- Pulse: 대기 큐에서 스레드 **1개**를 깨움
- PulseAll: 대기 큐의 스레드 **전부**를 깨움

Monitor.Wait / Pulse -- Reader

```
// Reader: 큐가 빌 때까지 대기  
while (Queue.Count == 0 && running)  
    Monitor.Wait(Gate);  
// 락 해제 후 대기, Pulse 시 재획득
```

- **Wait:** 락을 해제하고 대기 큐에 진입
- Pulse를 받으면 락을 다시 획득한 후 실행을 재개한다
- Producer-Consumer 패턴의 핵심 메커니즘

volatile 키워드 -- 최적화 억제

```
static volatile bool running = true;
```

- 컴파일러/JIT는 성능을 위해 변수 값을 레지스터에 캐싱하거나 명령어를 재배치할 수 있다
- `volatile` 은 이러한 최적화를 억제한다 (acquire/release fence 삽입)
- "메인 메모리 직접 접근"이 아님 -- 멀티프로세서에서 최신 값 보장도 아님 (MS 공식 문서)

volatile -- 비교표와 면접 포인트

	volatile 없음	volatile 있음
읽기	JIT가 레지스터 캐싱 가능	acquire fence -- 이 읽기 이후의 메모리 접근이 이 읽기보다 앞으로 재배치되지 않음
쓰기	재배치/지연 가능	release fence -- 이 쓰기 이전의 메모리 접근이 이 쓰기보다 뒤로 재배치되지 않음
원자성	보장 안 됨	보장 안 됨 (단일 읽기/쓰기는 32비트 이하라 원자적)

면접 TIP: "volatile은 메인 메모리에서 읽는 것인가?" -- **아니다**. volatile은 컴파일러/JIT/CPU의 최적화(레지스터 캐싱, 명령어 재배치)를 억제하는 **acquire/release fence**를 삽입하는 것이다. "메인 메모리 직접 접근"이라고 답하면 감점 요소가 될 수 있다.

Mutex -- 프로세스 간 동기화

```
using var mutex = new Mutex(false, @"Global\DemoMutex");  
mutex.WaitOne();    // 다른 프로세스가 가지면 대기  
try { /* 임계 구역 */ }  
finally { mutex.ReleaseMutex(); }
```

- Named Mutex를 사용하면 **프로세스 간 상호 배제**가 가능하다
- 커널 객체 기반이므로 lock 보다 느리다

Mutex vs lock (Monitor) 비교

	lock (Monitor)	Mutex
범위	같은 프로세스 내	프로세스 간 가능
구현	하이브리드 (경합 없으면 사용자 모드, 경합 시 커널)	커널 객체 (OS)
성능	빠름	느림 (커널 모드 전환)
이름 지정	불가	Global\ , Local\ 접두사

- 일반 스레드 동기화는 lock, 단일 인스턴스 앱/IPC는 Mutex

Semaphore -- 카운터 기반 동시 접근 제한

Mutex는 1개만 허용 + 스레드 소유권(획득한 스레드만 해제), Semaphore는 N개 허용 + 소유권 없음

```
// Named Semaphore: 동시 2개 스레드만 허용
using var sem = new Semaphore(2, 2, @"Global\SemDemo");
sem.WaitOne();    // 슬롯 1개 차지 (카운터 -1)
Thread.Sleep(1000);
sem.Release();    // 슬롯 반환 (카운터 +1)
```

- WaitOne(): 슬롯이 열릴 때까지 대기 (카운터가 0이면 블로킹)
- Release(): 사용 완료, 슬롯 반환
- 커널 기반이므로 프로세스 간 공유 가능

SemaphoreSlim vs Semaphore 비교

	Semaphore	SemaphoreSlim
구현	OS 커널 객체	.NET 관리형 (사용자 모드)
프로세스 간	가능 (Named)	불가
성능	느림	빠름
async 지원	없음	WaitAsync() 지원

- IPC가 필요하다면 Semaphore, 아니라면 SemaphoreSlim 우선

SemaphoreSlim -- 코드 예시

```
// SemaphoreSlim: async/await과 호환
using var slim = new SemaphoreSlim(2);
await slim.WaitAsync(); // 스레드 블로킹 없이 대기
try { /* 작업 */ }
finally { slim.Release(); }
```

- 같은 프로세스 내 비동기 제한에 최적
- `WaitAsync()` 는 스레드를 블로킹하지 않고 대기한다

Interlocked -- 원자 연산

Increment/Add는 lock xadd(x86) / LL/SC(ARM) 명령으로 수행, CompareExchange만 CAS(Compare-And-Swap) 사용

```
// lock 없이 Thread-Safe한 증가 (lock xadd)
Interlocked.Increment(ref _counter);
// CAS 패턴: 값이 예상값과 같을 때만 교체
Interlocked.CompareExchange(ref val, newVal, expected);
```

- 커널 모드 전환이 없다 -- 사용자 모드에서 완료
- Increment , Decrement , Add , Exchange , CompareExchange 제공
- 단순 연산만 가능 -- 복잡한 임계 구역에는 lock 사용

lock vs Interlocked 성능 비교

각 CPU 코어에서 500만 회 반복 증가 벤치마크 (Lab06):

방식	원리	경합 시 비용
lock	Monitor.Enter/Exit	커널 모드 전환 (수천 사이클)
Interlocked	CPU 원자적 명령 (lock xadd 등)	사용자 모드 완료 (수십 사이클)

```
lock (_gate) _lockCounter++; // 커널 전환  
Interlocked.Increment(ref _intCounter); // HW 원자 명령
```

정리: 단순 카운터/플래그 -> Interlocked, 복잡한 로직 -> lock

SpinLock -- busy-wait 방식

락을 얻을 때까지 스핀 대기 -- 컨텍스트 스위칭 비용 없음, 장기 경합 시 Thread.Yield 등으로 양보하는 적응형(adaptive) 스핀

```
static SpinLock spin = new();  
bool taken = false;  
spin.Enter(ref taken); // busy-wait으로 락 획득  
data++;  
if (taken) spin.Exit();
```

- 수 나노초 이하의 극히 짧은 임계 구역에 적합
- SpinLock 은 struct -- 복사하면 별도의 락이 된다

SpinLock -- 면접 포인트

면접 TIP: "SpinLock은 언제 쓰는가?" -- 경합이 드물고 임계 구역이 극히 짧을 때. 컨텍스트 스위칭 비용보다 spin 비용이 저렴한 경우에만 사용한다.

- 긴 작업에 사용하면 **CPU를 낭비**한다
- 반드시 **필드로 선언**하거나 `ref` 로 전달해야 한다 (struct 복사 방지)

ReaderWriterLockSlim -- 다중 리더 / 단일 라이터

읽기가 많고 쓰기가 드문 시나리오에서 lock 보다 효율적

```
static ReaderWriterLockSlim rw = new();  
rw.EnterReadLock();    // 읽기: 여러 스레드 동시 가능  
int val = data;  
rw.ExitReadLock();  
rw.EnterWriteLock();   // 쓰기: 독점  
data++;  
rw.ExitWriteLock();
```

락 모드	동시 읽기	동시 쓰기
ReadLock	허용	차단
WriteLock	차단	차단

AutoResetEvent vs ManualResetEvent

	AutoResetEvent	ManualResetEvent
비유	회전문 (1명 통과 후 자동 잠김)	수동 게이트 (열면 모두 통과)
Set()	대기자 1명만 통과, 자동 Reset	모든 대기자 통과
Reset	자동	수동 호출 필요
용도	1:1 신호 전달	1:N 브로드캐스트

AutoResetEvent / ManualResetEvent -- 코드

```
static AutoResetEvent auto = new(false);  
static ManualResetEvent manual = new(false);  
  
auto.Set();    // 대기자 1개만 깨움  
manual.Set();  // 모든 대기자가 통과  
manual.Reset();// 다시 비신호 상태로
```

- Auto: Set 호출 시 **하나만** 통과 후 자동으로 비신호 상태
- Manual: Reset을 명시적으로 호출해야 비신호 상태로 전환

ConcurrentQueue -- thread-safe (CAS 기반) 컬렉션

내부적으로 CAS 기반 알고리즘 사용 -- Thread-Safe

```
var queue = new ConcurrentQueue<int>();  
queue.Enqueue(value);           // lock 없이 안전하게 추가  
if (queue.TryDequeue(out int val))  
    Console.WriteLine(val);     // 원자적으로 꺼내기
```

- TryDequeue 는 확인과 꺼내기를 하나의 원자적 연산으로 수행
- Queue<T> 는 Thread-Safe하지 않으므로 반드시 ConcurrentQueue<T> 사용
- System.Collections.Concurrent 네임스페이스 소속

Actor 패턴 -- Channel 기반 메시지 패싱

공유 상태 없이, 각 Actor가 자신만의 상태와 메시지 큐를 가진다

```
class Actor
{
    readonly Channel<Message> _ch
        = Channel.CreateUnbounded<Message>();
    int _processedCount; // lock 불필요 -- 단일 루프 독점
}
```

- 이벤트 루프가 메시지를 순차 처리하므로 동기화 불필요
- 게임 서버에서 플레이어/룸 단위를 Actor로 모델링하면 lock 없이 상태 관리 가능

사용자 모드 vs 커널 모드 (1/2)

	사용자 모드	커널 모드
프리미티브	SpinLock , Interlocked	Mutex , Semaphore
성능	빠름 (모드 전환 없음)	느림 (커널 전환 필요)
프로세스 간	불가	가능 (Named 지원)

참고: lock (Monitor)은 경합 없으면 사용자 모드, 경합 시 스핀 대기 후 커널 모드로 전환하는 하이브리드이다.

사용자 모드 vs 커널 모드 (2/2)

	사용자 모드	커널 모드
CPU 사용	SpinLock은 busy-wait	블로킹 (OS가 관리)
async	SemaphoreSlim.WaitAsync	없음

- SemaphoreSlim 은 사용자 모드이며 WaitAsync 지원
- AutoResetEvent , ManualResetEvent 는 커널 모드

선택 기준: 같은 프로세스 내 -> 사용자 모드 우선, 프로세스 간 -> 커널 모드 필수

고찰점 -- 게임 서버 동기화 전략 (1/2)

- 단순 카운터 (HP, 골드): `Interlocked`
- 인벤토리 수정 (복합 로직): `lock`
- 읽기 위주 캐시 (설정, 랭킹 조회): `ReaderWriterLockSlim`

고찰점 -- 게임 서버 동기화 전략 (2/2)

- 동시 접속 제한 (방 입장 인원): SemaphoreSlim
- 플레이어 상태 관리 (독립 처리): Actor 패턴 (Channel)
- 프로세스 간 자원 (서버 단일 실행): Mutex

면접 TIP: "동기화 기법을 선택하는 기준은?" -- (1) 임계 구역의 길이, (2) 경합 빈도, (3) 프로세스 간 필요 여부, (4) async 호환성을 종합적으로 판단한다. 가장 가벼운 수단부터 적용하는 것이 원칙이다.

실습 안내 (1/2)

Lab	주제	핵심 프리미티브
Lab01	Race Condition & lock	lock , Monitor
Lab02	Deadlock 재현 및 해결	Lock Ordering
Lab03	Producer-Consumer	Monitor.Wait/Pulse
Lab04	Named Mutex	Mutex (프로세스 간)

실습 안내 (2/2)

Lab	주제	핵심 프리미티브
Lab05	Semaphore / SemaphoreSlim	Semaphore , SemaphoreSlim
Lab06	Interlocked 벤치마크	Interlocked , CAS
Lab07	SpinLock & RW Lock	SpinLock , ReaderWriterLockSlim
Lab08	Auto/ManualResetEvent	이벤트 신호

- Lab09: **ConcurrentQueue** -- Thread-Safe (CAS 기반) 컬렉션
- Lab10: **Actor 패턴** -- Channel<T>

핵심 정리 (1/2)

1. Race Condition은 공유 데이터의 비원자적 접근에서 발생 -- lock 으로 해결
2. Deadlock은 4가지 조건이 모두 충족될 때 발생 -- 락 순서 통일로 예방
3. volatile은 최적화 억제(acquire/release fence)이지 "메인 메모리 접근"이 아님, 원자성은 Interlocked
4. lock vs Interlocked: 복잡한 로직은 lock, 단순 연산은 Interlocked
5. Mutex/Semaphore: 프로세스 간 동기화에는 커널 객체 필수

핵심 정리 (2/2)

- 6. **SemaphoreSlim**: 같은 프로세스 내라면 `WaitAsync` 지원하는 경량 버전 우선
- 7. **SpinLock**: 극히 짧은 임계 구역에서만, struct 복사 주의
- 8. **ReaderWriterLockSlim**: 읽기 다수/쓰기 소수 패턴에 최적
- 9. **ConcurrentQueue**: CAS 기반 thread-safe 컬렉션
- 10. **Actor 패턴**: 공유 상태를 제거하고 메시지로 통신 -- 가장 안전한 설계